

# Complete Notes

## PHP `\$\_GET` and `\$\_POST` Explained: Comprehensive Study Notes

### 1. Executive Summary

This document provides a detailed explanation of PHP's `\$\_GET` and `\$\_POST` superglobal variables, which are fundamental for handling data submitted via HTTP requests, typically from HTML forms. We will explore their core definitions, how they differ in terms of data transmission and visibility, their respective use cases, and important security considerations. Understanding these superglobals is crucial for any PHP developer building interactive web applications that collect user input.

### 2. Key Concepts

- **HTTP Request Methods:**

The primary ways a client (e.g., a web browser) communicates with a server. The two most common methods for sending data are GET and POST.

- **Superglobal Variables:**

Special built-in PHP variables that are always available in all scopes throughout a script. They do not require `global` keyword to be accessed inside functions or methods. `\$\_GET` and `\$\_POST` are examples of superglobals.

- **Form Submission:**

The process by which user-entered data from an HTML form is sent to a web server for processing. The `method` attribute of the `` tag determines whether GET or POST is used.

- **URL Parameters (Query String):**

Data appended to a URL after a question mark (`?`). Each parameter is a key-value pair, separated by an ampersand (`&`). This is how data is transmitted using the GET method.

- **Request Body:**

The part of an HTTP request that contains the actual data being sent from the client to the server. This is where data is transmitted using the POST method, making it invisible in the URL.

- **Idempotence:**

An operation is idempotent if it produces the same result no matter how many times it is executed. GET requests are generally considered idempotent (retrieving data multiple times doesn't change server state), while POST requests are not (submitting a form multiple times might create multiple entries).

- **Security Considerations:**

The choice between GET and POST has implications for data privacy (visibility in URL) and data integrity (how data is handled).

### 3. Core Definitions

- **`\$\_GET`:**
  - A **PHP superglobal associative array** used to collect data sent with the **HTTP GET method**.
  - Data is appended to the URL as a **query string** (e.g., `page.php?name=John&age=30`).
  - **Visibility:** Data is visible in the browser's URL bar and browser history.
  - **Data Limit:**  
Has practical limits on the amount of data that can be sent (typically around 2048 characters, depending on the browser and server).
- **Use Cases:** Retrieving data, search queries, pagination, filtering, bookmarkable links.
- **Security:** Not suitable for sensitive data (passwords, personal info) due to URL visibility.
- **Idempotent:**  
Generally considered idempotent, meaning repeated requests should not have additional side effects.
- **`\$\_POST`:**
  - A **PHP superglobal associative array** used to collect data sent with the **HTTP POST method**.
  - Data is sent in the **HTTP request body**, not in the URL.
  - **Visibility:** Data is not visible in the browser's URL bar or browser history.
  - **Data Limit:**  
Has no practical limits on the amount of data that can be sent (limited by server configuration, e.g., `post_max_size` in `php.ini`).
- **Use Cases:**  
Submitting forms (login, registration, contact forms), uploading files, sending large amounts of data, operations that modify server state.
- **Security:** More suitable for sensitive data as it's not exposed in the URL.
- **Not Idempotent:**  
Repeated requests can have additional side effects (e.g., submitting the same order multiple times).
- **Superglobal Variable**  
: A predefined PHP variable that is always accessible, regardless of the current scope. Examples include ``$_GET``, ``$_POST``, ``$_SESSION``, ``$_COOKIE``, ``$_SERVER``, ``$_FILES``, ``$_REQUEST``, ``$_ENV``, ``$_GLOBALS``.

### 4. Examples

#### Example 1: Using ``$_GET``

**Scenario:** A simple search form where the search term is passed via the URL.

**HTML Form (`search.html`):**

```
```html
<!DOCTYPE html>
```

```

<html lang="en">
<head>
<meta charset="UTF-8">
<title>Search Page</title>
</head>
<body>
<h1>Search for Products</h1>
<form action="results.php" method="get">
<label for="query">Search Term:</label>
<input type="text" id="query" name="search_query" required>
<button type="submit">Search</button>
</form>
</body>
</html>
...

```

### PHP Script (`results.php`):

```

```php
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Search Results</title>
</head>
<body>
<h1>Search Results</h1>
<?php
// Check if the 'search_query' parameter exists in the URL
if (isset($_GET['search_query'])) {
$searchTerm = htmlspecialchars($_GET['search_query']); // Sanitize input
echo "<p>You searched for: <strong>" . $searchTerm . "</strong></p>";
echo "<p>Displaying results for '" . $searchTerm . "'...</p>";
// In a real application, you would query a database here
} else {
echo "<p>No search term provided.</p>";
}
?>
<p><a href="search.html">Go back to search</a></p>
</body>
</html>
...

```

### How it works:

1. When you submit `search.html` with "laptop" in the search box, the browser navigates to `results.php?search\_query=laptop`.
2. The `results.php` script accesses the `search\_query` value using `\$\_GET['search\_query']`.

## Example 2: Using `\$\_POST`

**Scenario:** A contact form where user details are submitted privately.

### HTML Form (`contact.html`):

```
```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Contact Us</title>
</head>
<body>
<h1>Contact Us</h1>
<form action="submit_contact.php" method="post">
<label for="name">Your Name:</label><br>
<input type="text" id="name" name="user_name" required><br><br>

<label for="email">Your Email:</label><br>
<input type="email" id="email" name="user_email" required><br><br>

<label for="message">Your Message:</label><br>
<textarea id="message" name="user_message" rows="5" required></textarea><br><br>

<button type="submit">Send Message</button>
</form>
</body>
</html>
```
```

### PHP Script (`submit\_contact.php`):

```
```php
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Contact Submission</title>
</head>
<body>
<h1>Thank You for Your Message!</h1>
<?php
// Check if the form was submitted using POST
if ($_SERVER["REQUEST_METHOD"] == "POST") {
// Retrieve and sanitize data from $_POST
$name = htmlspecialchars($_POST['user_name']);
$email = htmlspecialchars($_POST['user_email']);
$message = htmlspecialchars($_POST['user_message']);
```

```

echo "<p>Name: <strong>" . $name . "</strong></p>";
echo "<p>Email: <strong>" . $email . "</strong></p>";
echo "<p>Message: <strong>" . $message . "</strong></p>";
echo "<p>Your message has been received. We will get back to you shortly.</p>";

// In a real application, you would save this data to a database
// or send an email.
} else {
echo "<p>This page should be accessed via a POST request from the contact form.</p>";
}
?>
<p><a href="contact.html">Go back to contact form</a></p>
</body>
</html>
...

```

### How it works:

1. When you submit `contact.html`, the data is sent to `submit\_contact.php` in the HTTP request body.
2. The URL remains `submit\_contact.php`, without any visible parameters.
3. The `submit\_contact.php` script accesses the form data using `\$\_POST['user\_name']`, `\$\_POST['user\_email']`, and `\$\_POST['user\_message']`.

## 5. Practice Questions

1. **What is the primary difference between how `\$\_GET` and `\$\_POST` transmit data to a PHP script?**

- A) `\$\_GET` uses cookies, `\$\_POST` uses sessions.
- B) `\$\_GET` appends data to the URL, `\$\_POST` sends data in the HTTP request body.
- C) `\$\_GET` is for text, `\$\_POST` is for files.
- D) `\$\_GET` is for internal use, `\$\_POST` is for external use.

2.

**Which PHP superglobal variable would you use to retrieve data from a form submitted with `method="post"`?**

- A) `\$\_REQUEST`
- B) `\$\_SESSION`
- C) `\$\_GET`
- D) `\$\_POST`

3. **True or False:**

Data sent via the GET method is generally considered more secure for sensitive information like passwords because it's encrypted.

4.

**You are building a search page where users can search for products. Which HTTP method (`GET` or `POST`) would be more appropriate for the search form, and why?**

5. **Consider the following HTML form:**

```
```html
<form action="process.php" method="get">
<input type="text" name="item_id" value="123">
<input type="submit" value="View Item">
</form>
```
```

If a user submits this form, what would the URL look like, and how would you access the `item\_id` in `process.php`?

6.  
**What is a "superglobal variable" in PHP, and why are `\$\_GET` and `\$\_POST` considered superglobals?**

7.  
**When might you encounter a "data limit" issue, and which method (`GET` or `POST`) is more susceptible to it?**

**8. Scenario:**

A user fills out a registration form with their username, email, and password. Which method (`GET` or `POST`) should be used for submitting this form, and what are the security implications of choosing the wrong method?

---

**Answer Key:**

1. **B)** `\$\_GET` appends data to the URL, `\$\_POST` sends data in the HTTP request body.

2. **D)** `\$\_POST`

3. **False.**

Data sent via GET is visible in the URL and is not inherently encrypted. POST is generally preferred for sensitive data because it's not exposed in the URL.

4. **GET** would be more appropriate.

• **Why:**

Search results are often bookmarkable and shareable. The search query in the URL allows users to easily share specific search results or refresh the page without re-entering the query. GET requests are also generally idempotent, meaning repeating the search doesn't change server state.

5. **URL:** `process.php?item\_id=123`

**Access in `process.php`:** `echo \$\_GET['item\_id'];`

6. A **superglobal variable**

is a predefined PHP variable that is always accessible in all scopes throughout a script, without needing to use the `global` keyword. `\$\_GET` and `\$\_POST` are superglobals because they are automatically available everywhere in your code to collect HTTP request data.

7. You might encounter a data limit issue when sending a very large amount of data. The **GET** method is more susceptible to this, as URLs have practical length limits (e.g., around 2048 characters in many browsers). The POST method has much larger limits, typically configured by the server's `post\_max\_size` setting.

8. The **POST** method should be used.

- **Security Implications of choosing GET:**

If GET were used, the username, email, and especially the password would be visible directly in the browser's URL bar, in browser history, and in server logs. This makes the sensitive information easily accessible to anyone looking over the user's shoulder, or if the URL is accidentally shared or logged, leading to severe security vulnerabilities. POST hides this data in the request body, making it much more secure for sensitive information.